Code Walkthrough Guide

Which folders to open when presenting, which to mention, and which to leave alone.

Cloud-Based Auditing System with Efficient Ownership Management for Group Data Transactions · Dept. of ISE, Global Academy of Technology

The project contains roughly 6,000 lines of Python across seven folders. Only three of them carry the story. This guide ranks every folder by how useful it is in front of an audience, so that presentation time goes to the material that earns attention.

Tier 1: open these, this is the project

OPEN `crypto/` the protocol itself

If anyone asks where the research is actually implemented, this is the answer. It contains no file handling, no network code and no database code. Pure cryptography, readable on its own.

FILE	LINES	WHAT TO SAY ABOUT IT
`scheme.py`	397	**The single most important file.** Contains SysGen, KeyGen, TagGen and Audit, four of the six algorithms.
`ownership.py`	354	**The paper's actual contribution.** AddOwner, RevokeOwner and OwnerTransfer. The security trick lives here.
`hashes.py`	120	Short, and the place where two errors in the published paper were identified and corrected.
`pairing.py`	265	The mathematics engine. Worth noting it is the only file that touches the cryptography library, so the whole backend is swappable in one move.

OPEN `attacks/` the strongest material in the project

Three attacks, each run twice: once against the older published method, once against this one. Demonstrating only the defence would prove nothing, since a system that rejects everything also rejects attacks.

FILE	LINES	WHAT IT SHOWS
`collusion.py`	260	**Run this one live.** Breaks the older method, then blocks the identical attack.
`rogue_key.py`	247	Faked co-ownership of a stranger's dataset, blocked.
`tampering.py`	253	Corruption detection, plus the 3,500-trial statistical study.

RUN `demo/run_demo.py` 338 lines

The narrated walkthrough that gets run in front of people. It plays out the full lifecycle on a real 25 MB dataset and prints the auditor's verdict at every stage.

Tier 2: mention, open only if asked

MENTION `entities/` the four parties

Cloud Storage Provider, Third Party Auditor, Sale Group and Buy Group, written as plain classes. Useful for explaining who is trusted with what. **`csp.py`** is the interesting one, because it deliberately includes methods to corrupt and delete blocks. That is the modelled attack surface, put there on purpose.

MENTION `tests/` 542 lines

Do not read this aloud. Simply state that there are 41 tests and all of them pass, then show the terminal output instead.

SHOW OUTPUT `benchmarks/`

Point at the charts rather than the code. The six result images sit in **`benchmarks/results/figures/`**.

Tier 3: only if someone digs

> **`datasets/`** generated the 500 MB of sample data. **`crypto/baseline.py`** is the comparison system used for benchmarking. **`crypto/serialize.py`** and **`demo/console.py`** are utilities. All solid work, but none of it wins an argument.

Do not open these in front of anyone. **`benchmarks/plots.py`** (509 lines) and **`datasets/generate.py`** (552 lines) are the two largest files in the project and the two least interesting. One is chart styling, the other is synthetic data generation. Opening them costs presentation time and impresses nobody.

If there are only 60 seconds

Show two folders and say one line about each.

FOLDER	THE LINE
`code/crypto/`	"This is the research paper, implemented."
`code/attacks/`	"This is the proof that it works."

If there are five minutes, do exactly this

1. **Run it rather than describe it.** The audience watches the older method break and this one hold. Considerably stronger than any slide.

   ```
   $ python3 -m attacks.collusion
   ```

2. **Open `crypto/ownership.py`** and scroll to the comment block at the top. It explains the three deliberate design decisions in plain English before any code appears.
3. **Show `benchmarks/results/figures/fig1_keygen_scaling.png`.** The two diverging curves make the scalability point instantly, with no explanation required.

Two points worth saying out loud

On the architecture. The dependency direction is strictly one way. **`crypto/`** does not know that **`entities/`** exists, and **`entities/`** does not know that **`demo/`** exists. That was a deliberate decision, and it is the reason the cryptography can be read on its own and found complete.

On the comments. Every file opens with an explanation of why it exists, not merely what it does, including the corrections made to the published paper. Open any file at random and the first thing read is reasoning, not code.

Anticipated questions, and where to point

QUESTION	ANSWER, AND THE FILE TO OPEN
"Where is the actual research implemented?"	**`crypto/scheme.py`** and **`crypto/ownership.py`**
"How do you know it is secure?"	Run **`attacks/collusion.py`** live
"How do you know it is correct?"	`pytest tests/ -v`, 41 tests
"Did you just copy the authors' code?"	No. Written from the equations. Their repository is cited as related work in **`README.md`**; two errors in their notation were corrected in **`crypto/hashes.py`**
"Why is it slower than the paper?"	A deliberate choice of pure-Python so it runs anywhere without a compiler. Affects raw speed only, not the scaling result. See **`README.md`**
"Is the sample data real?"	No, entirely synthetic. No real patient, customer or personal record is used. See **`datasets/generate.py`**